

Лабораторная работа № 4. Линейная алгебра

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Поэлементные операции над многомерными массивами	7
3.2	Транспонирование, след, ранг, определитель и инверсия матрицы	8
3.3	Вычисление нормы векторов и матриц, повороты, вращения . . .	9
3.4	Матричное умножение, единичная матрица, скалярное произведе- ние	11
3.5	Факторизация. Специальные матричные структуры	11
3.6	Общая линейная алгебра	19
3.7	Задания для самостоятельного выполнения	20
3.7.1	Произведения векторов	20
3.7.2	Системы линейных уравнений	21
3.7.3	Операции с матрицами	23
3.7.4	Линейные модели экономики	25
4	Выводы	29

Список иллюстраций

3.1	Примеры поэлементных операций	8
3.2	Различные операции над матрицами	9
3.3	Вычисления нормы векторов и матриц, повороты, вращения . . .	10
3.4	Примеры матричного умножения, единичной матрицы, скалярно- го произведения	11
3.5	Примеры работы со специальными матричными структурами . .	12
3.6	Решение исходной системы уравнений	14
3.7	Примеры работы с матрицами большой размерности и специаль- ной структуры	16
3.8	Оценка эффективности	18
3.9	Оценка эффективности	19
3.10	Пример решения системы линейных уравнений с рациональными элементами	20
3.11	Пункт 1 и пункт 2	21
3.12	Решение СЛАУ с двумя неизвестными	22
3.13	Решение СЛАУ с тремя неизвестными	23
3.14	Приведение матриц к диагональному виду и вычисление	24
3.15	Создание диагональной матрицы из собственных значений, ниж- недиагональной матрицы, оценка эффективности	25
3.16	Создание функций	26
3.17	Проверка с помощью определения и критерия на продуктивность	27
3.18	Проверка с помощью спектрального критерия	28

Список таблиц

1 Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

2 Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

3 Выполнение лабораторной работы

3.1 Поэлементные операции над многомерными массивами

Для матрицы 4×3 рассмотрим поэлементные операции сложения и произведения её элементов. Для работы со средними значениями можно воспользоваться возможностями пакета Statistics (рис. 3.1):

Поэлементные операции над многомерными массивами

```
[1]: # Массив 4x3 со случайными целыми числами (от 1 до 20):
a = rand(1:20,(4,3))

[1]: 4x3 Matrix{Int64}:
 4 19  6
 8  5  5
 3 14 12
16 18 17

[3]: # Поэлементная сумма:
sum(a)

[3]: 127

[4]: # Поэлементная сумма по столбцам:
sum(a,dims=1)

[4]: 1x3 Matrix{Int64}:
31 56 40

[5]: # Поэлементная сумма по строкам:
sum(a,dims=2)

[5]: 4x1 Matrix{Int64}:
29
18
29
51

[6]: # Поэлементное произведение:
prod(a)

[6]: 225843608800

[7]: # Поэлементное произведение по столбцам:
prod(a,dims=1)

[7]: 1x3 Matrix{Int64}:
1536 23940 6120

[8]: # Поэлементное произведение по строкам:
prod(a,dims=2)

[8]: 4x1 Matrix{Int64}:
456
200
504
4896

[9]: using Statistics

[10]: # Вычисление среднего значения массива:
mean(a)

[10]: 10.583333333333334

[11]: # Среднее по столбцам:
mean(a,dims=1)

[11]: 1x3 Matrix{Float64}:
7.75 14.0 10.0

[12]: # Среднее по строкам:
mean(a,dims=2)

[12]: 4x1 Matrix{Float64}:
9.666666666666666
6.0
9.666666666666666
17.0
```

Рис. 3.1: Примеры поэлементных операций

3.2 Транспонирование, след, ранг, определитель и инверсия матрицы

Для выполнения таких операций над матрицами, как транспонирование, диагонализация, определение следа, ранга, определителя матрицы и т.п. можно воспользоваться библиотекой (пакетом) LinearAlgebra (рис. 3.2):

```

Транспонирование, след, ранг, определитель и инверсия матрицы

using LinearAlgebra

# Массив 4x4 со случайными целыми числами (от 1 до 20):
b = rand(1:20, (4,4))

4x4 Matrix{Int64}:
 5  9  5 12
 8  2 10 11
 6  7 10 11
14 16 10  9

# Транспонирование:
transpose(b)

4x4 transpose{::Matrix{Int64}} with eltype Int64:
 5  8  6 14
 9  2  7 16
 5 10 10 10
12 11 11  9

# След матрицы (сумма диагональных элементов):
tr(b)

26

# Извлечение диагональных элементов как массив:
diag(b)

4-element Vector{Int64}:
 5
 2
10
 9

# Ранг матрицы:
rank(b)

4

# Инверсия матрицы (определение обратной матрицы):
inv(b)

4x4 Matrix{Float64}:
 0.0238663  0.165871 -0.25537  0.0775656
 0.00954654 -0.133652  0.097852  0.0310263
 -0.17327  -0.0742243  0.273986 -0.0131265
 0.138425  0.0620525 -0.0811456 -0.0501193

# Определитель матрицы:
det(b)

4190.000000000001

# Псевдообратная функция для прямоугольных матриц:
pinv(a)

3x4 Matrix{Float64}:
 0.0114402  0.0649661 -0.097037  0.0453513
 0.080733  -0.00130902 -0.016552 -0.0164252
 -0.09665  -0.0371761  0.117903  0.0206437

```

Рис. 3.2: Различные операции над матрицами

3.3 Вычисление нормы векторов и матриц, повороты, вращения

Евклидова норма:

$$\|\vec{X}\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

p-норма:

$$\|\vec{A}\|_p = \left(\sum_{i=1}^n |a_i|^p \right)^{1/p}$$

Вычисление нормы векторов и матриц, повороты, вращения

```
[22]: # Создание вектора X:
X = [2, 4, -5]

[22]: 3-element Vector{Int64}:
      2
      4
     -5

[23]: # Вычисление евклидовой нормы:
norm(X)

[23]: 6.708203932499369

[24]: # Вычисление p-нормы:
p = 1
norm(X,p)

[24]: 11.0

[25]: # Расстояние между двумя векторами X и Y:
X = [2, 4, -5];
Y = [1, -1, 3];
norm(X-Y)

[25]: 9.486832988505138

[26]: # Проверка по базовому определению:
sqrt(sum((X-Y).^2))

[26]: 9.486832988505138

[27]: # Угол между двумя векторами:
acos((transpose(X)*Y)/(norm(X)*norm(Y)))

[27]: 2.4404307889469252

[28]: # Создание матрицы:
d = [5 -4 2; -1 2 3; -2 1 0]

[28]: 3x3 Matrix{Int64}:
      5  -4  2
     -1   2  3
     -2   1  0

[29]: # Вычисление Евклидовой нормы:
opnorm(d)

[29]: 7.147682841795258

[30]: # Вычисление p-нормы:
p=1
opnorm(d,p)

[30]: 8.0

[31]: # Поворот на 180 градусов:
rot180(d)

[31]: 3x3 Matrix{Int64}:
      0   1  -2
      3   2  -1
      2  -4   5

[32]: # Переворачивание строк:
reverse(d,dims=1)

[32]: 3x3 Matrix{Int64}:
     -2   1  0
     -1   2  3
      5  -4  2

[33]: # Переворачивание столбцов
reverse(d,dims=2)

[33]: 3x3 Matrix{Int64}:
      2  -4   5
      3   2  -1
      0   1  -2
```

Рис. 3.3: Вычисления нормы векторов и матриц, повороты, вращения

3.4 Матричное умножение, единичная матрица, скалярное произведение

Рассмотрим следующие примеры в данном разделе (рис. 3.4):

Матричное умножение, единичная матрица, скалярное произведение

```
[34]: # Матрица 2x3 со случайными целыми значениями от 1 до 10:  
A = rand(1:10,(2,3))  
[34]: 2x3 Matrix{Int64}:  
5 8 7  
2 9 7  
[35]: # Матрица 3x4 со случайными целыми значениями от 1 до 10:  
B = rand(1:10,(3,4))  
[35]: 3x4 Matrix{Int64}:  
10 9 3 2  
8 8 5 6  
3 2 10 8  
[36]: # Произведение матриц A и B:  
A*B  
[36]: 2x4 Matrix{Int64}:  
135 123 125 114  
113 104 121 114  
[37]: # Единичная матрица 3x3:  
Matrix{Int}(I, 3, 3)  
[37]: 3x3 Matrix{Int64}:  
1 0 0  
0 1 0  
0 0 1  
[38]: # Скалярное произведение векторов X и Y:  
X = [2, 4, -5]  
Y = [1, -1, 3]  
dot(X,Y)  
[38]: -17  
[39]: # тоже скалярное произведение:  
X*Y  
[39]: -17
```

Рис. 3.4: Примеры матричного умножения, единичной матрицы, скалярного произведения

3.5 Факторизация. Специальные матричные структуры

В математике факторизация (или разложение) объекта — его декомпозиция (например, числа, полинома или матрицы) в произведение других объектов или факторов, которые, будучи перемноженными, дают исходный объект.

Рассмотрим несколько примеров. Для работы со специальными матричными структурами потребуется пакет `LinearAlgebra` (рис. 3.5):

[39]: -17

Факторизация. Специальные матричные структуры

```
[40]: # Задаём квадратную матрицу 3x3 со случайными значениями:  
A = rand(3, 3)
```

```
[40]: 3x3 Matrix{Float64}:  
 0.932133  0.375629  0.348507  
 0.853367  0.52272  0.236673  
 0.771063  0.346031  0.131079
```

```
[41]: # Задаём единичный вектор:  
x = fill(1.0, 3)
```

```
[41]: 3-element Vector{Float64}:  
 1.0  
 1.0  
 1.0
```

```
[42]: # Задаём вектор b:  
b = A*x
```

```
[42]: 3-element Vector{Float64}:  
 1.6562692355675925  
 1.6127594578242905  
 1.2481732826412582
```

```
[43]: # Решению исходного уравнения получаем с помощью функции \  
# (убеждаемся, что x - единичный вектор):  
A\b
```

```
[43]: 3-element Vector{Float64}:  
 0.9999999999999988  
 1.0000000000000016  
 1.0000000000000001
```

```
[44]: # LU-факторизация:  
Alu = lu(A)
```

```
[44]: LU{Float64, Matrix{Float64}, Vector{Int64}}  
L factor:  
3x3 Matrix{Float64}:  
 1.0      0.0      0.0  
 0.915499  1.0      0.0  
 0.827203  0.197448  1.0  
U factor:  
3x3 Matrix{Float64}:  
 0.932133  0.375629  0.348507  
 0.0      0.178832  -0.0823853  
 0.0      0.0      -0.14094
```

```
[45]: # Матрица перестановок:  
Alu.P
```

```
[45]: 3x3 Matrix{Float64}:  
 1.0  0.0  0.0  
 0.0  1.0  0.0  
 0.0  0.0  1.0
```

```
[46]: # Вектор перестановок:  
Alu.p
```

```
[46]: 3-element Vector{Int64}:  
 1  
 2  
 3
```

```
[47]: # Матрица L:  
Alu.L
```

```
[47]: 3x3 Matrix{Float64}:  
 1.0      0.0      0.0  
 0.915499  1.0      0.0  
 0.827203  0.197448  1.0
```

```
[48]: # Матрица U:  
Alu.U
```

```
[48]: 3x3 Matrix{Float64}:  
 0.932133  0.375629  0.348507  
 0.0      0.178832  -0.0823853  
 0.0      0.0      -0.14094
```

Рис. 3.5: Примеры работы со специальными матричными структурами

Исходная система уравнений $Ax = b$ может быть решена или с использованием исходной матрицы, или с использованием объекта факторизации (рис. 3.6):

```

[49]: # Решение СЛАУ через матрицу A:
A\b

[49]: 3-element Vector{Float64}:
 0.9999999999999988
 1.0000000000000016
 1.0000000000000001

[50]: # Решение СЛАУ через объект факторизации:
Alu\b

[50]: 3-element Vector{Float64}:
 0.9999999999999988
 1.0000000000000016
 1.0000000000000001

[52]: # Детерминант матрицы A:
det(A)

[52]: -0.023494138588288225

[53]: # Детерминант матрицы A через объект факторизации:
det(Alu)

[53]: -0.023494138588288225

[54]: # QR-факторизация:
Aqr = qr(A)

[54]: LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
Q factor: 3x3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
R factor:
3x3 Matrix{Float64}:
-1.48042 -0.718052 -0.424132
 0.0 -0.135909 0.8336167
 0.0 0.0 -0.116768

[55]: # Матрица Q:
Aqr.Q

[55]: 3x3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}

[56]: # Матрица R:
Aqr.R

[56]: 3x3 Matrix{Float64}:
-1.48042 -0.718052 -0.424132
 0.0 -0.135909 0.8336167
 0.0 0.0 -0.116768

[57]: # Проверка, что матрица Q - ортогональная:
Aqr.Q'*Aqr.Q

[57]: 3x3 Matrix{Float64}:
 1.0 0.0 -1.11022e-16
 0.0 1.0 0.0
 0.0 0.0 1.0

[58]: # Симметризация матрицы A:
Asym = A + A'

[58]: 3x3 Matrix{Float64}:
 1.06427 1.229 1.11957
 1.229 1.04544 0.582704
 1.11957 0.582704 0.262158

[59]: # Спектральное разложение симметризованной матрицы:
AsymEig = eigen(Asym)

[59]: Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
3-element Vector{Float64}:
-0.3161862982418005
 0.21746724709618592
 3.270583720610219
vectors:
3x3 Matrix{Float64}:
-0.483761 0.430914 -0.761766
 0.0630746 -0.850962 -0.521426
 0.872924 0.300294 -0.384483

[60]: # Собственные значения:
AsymEig.values

[60]: 3-element Vector{Float64}:
-0.3161862982418005
 0.21746724709618592
 3.270583720610219

```

Рис. 3.6: Решение исходной системы уравнений

Далее рассмотрим примеры работы с матрицами большой размерности и специальной структуры (рис. 3.7):

```

[61]: #Собственные векторы:
      AsymEig.vectors

[61]: 3x3 Matrix(Float64):
      -0.483761  0.438914 -0.761766
      0.0638746 -0.858962 -0.521426
      0.872924  0.380294 -0.384483

[62]: # Проверим, что получится единичная матрица:
      Inv(AsymEig)*Asym

[62]: 3x3 Matrix(Float64):
      1.0      1.55431e-15  3.33867e-16
      -1.33227e-15  1.0      6.66134e-16
      1.33227e-15 -2.22845e-16  1.0

[64]: # Матрица 1000 x 1000:
      n = 1000
      A = randn(n,n)

[64]: 1000x1000 Matrix{Float64}:
      -0.818448  1.26829 -0.358636  - -2.13164 -1.45814 -1.46844
      -0.188574  1.56135  0.112782  -0.361828 -0.283565  0.781482
      -1.06557  1.32281  0.345321  -1.08543  1.88505 -0.888855
      -1.65216 -0.895826  0.682588  -1.81485 -0.298681 -1.7884
      -0.916876  0.348518  0.8316399  0.8566955  1.15914 -1.29557
      -1.47983 -0.6529 -0.8212892  - -1.11195 -0.8223531  1.89988
      0.695698  1.08955 -1.15135 -2.83542  0.565761  1.14
      -0.299825  0.851777 -0.8786415  0.659225  0.188935  1.32651
      -0.249889 -1.01681 -0.828267  1.21828 -0.8199867  0.8667793
      -0.8277812  0.569627 -3.91664 -0.387882 -0.328859 -0.82485
      -2.65577  0.818296 -0.8452734 -0.247238 -0.254913 -1.66853
      0.987181  0.0678187 -0.511481  0.781657  0.837263  0.896972
      1.38147 -0.789347  0.614746  0.8281318  0.651188 -0.569171
      ⋮
      -0.383486 -0.1881  0.341371 -0.8188559  0.136223 -0.21117
      1.87889  1.23558  1.72797 -1.36745  0.178284 -1.84882
      -0.653328  0.923418 -1.45991 -0.411953  0.588842 -0.495116
      0.486941 -1.84535 -0.652696 -1.72691 -0.348355 -0.369854
      -0.391725 -0.356184  1.14266  1.95623  1.33086 -0.111696
      2.0482  0.580253 -0.793761  1.97551  1.14517 -0.789763
      -0.664224  0.864611  2.18768  0.833937 -0.245346  1.1135
      -1.13872  0.128874 -0.378284 -0.833325  0.111267 -1.76335
      -1.23381 -1.77842 -0.611632  0.389125 -1.32341 -0.878754
      -0.127285  0.28557  0.648935 -0.538279 -0.128879  0.786378
      0.675187  0.498474 -0.88895962 -1.25196 -0.457293  2.57739
      -0.427728  0.12144  1.05115  0.8339976 -2.58434  0.541753

[65]: # Симметризация матрицы:
      Asym = A + A'

[65]: 1000x1000 Matrix{Float64}:
      -1.6289  1.07971 -1.41621 -2.71769 - -0.774953 -1.88817
      1.07971  3.1227  1.43551  0.8216555  0.286989  0.982922
      -1.41621  1.43551  0.698643  1.6782  1.87689  0.242296
      -2.71769  0.8216555  1.6782  2.44286 -0.634897 -2.83436
      -0.89372 -0.694587  0.713968 -0.458486  2.23828 -2.3284
      -1.84811  0.733881 -2.82818 -2.64185 -0.8948168  1.53355
      0.54638  1.359  1.24886  0.54343  1.81811  0.945984
      -1.44939  0.827187  1.3188  0.679938  1.55354  0.347781
      1.54457 -0.837363 -1.31887 -1.56792 -0.271563 -1.87729
      -0.215787  0.511466 -2.69192  0.26919  0.8336246 -0.319774
      -4.31362  1.24963  1.25732  0.855534 -1.37662 -3.14565
      1.38665  2.83196 -2.52662  0.8668982 -0.633672  0.3982
      1.72119 -0.512198  1.22839  0.175869 -0.883392 -1.38454
      ⋮
      -0.819445 -0.591849 -1.32786  2.69286  0.297423  0.388347
      2.79525  1.29827  3.35323 -2.14452 -0.942964 -2.7884
      -0.768885  1.86249 -1.33621 -0.188748 -2.83895 -0.168381
      -0.685617 -0.982597 -1.35331 -3.1714 -0.675688  0.238783
      -0.75288  0.8728725  2.87122 -0.714824  1.87543  0.8287712
      3.8112  2.11567  0.926898 -0.695955  0.666853 -1.65378
      0.495185  2.78711  3.27376  0.767181 -0.562891  2.13788
      -1.18185  0.729133 -0.592985 -2.4141 -0.518696 -0.634844
      -2.23668 -2.9584  0.154699 -1.71485 -0.578891 -0.543797
      -2.25884 -0.8762578 -0.4445 -0.556246 -1.38884  0.828376
      -0.774953  0.286989  1.87689 -0.634897 -0.914585 -0.88694849
      -1.88817  0.982922  0.242296 -2.83436 -0.88694849  1.88351

[66]: # Проверка, является ли матрица симметричной:
      issymmetric(Asym)

[66]: true

[67]: # Добавление шума:
      Asym_noisy = copy(Asym)
      Asym_noisy[1,2] += Seps()

[67]: 1.0797134624244016

```

Рис. 3.7: Примеры работы с матрицами большой размерности и специальной структуры

Далее для оценки эффективности выполнения операций над матрицами большой размерности и специальной структуры воспользуемся пакетом BenchmarkTools (рис. 3.8):

```

[68]: # Проверка, является ли матрица симметричной:
issymmetric(Asym_noisy)

[68]: false

[69]: # Явно указываем, что матрица является симметричной:
Asym_explicit = Symmetric(Asym_noisy)

[69]: 1000x1000 Symmetric{Float64, Matrix{Float64}}:
-1.6209  1.07971 -1.41621 -2.71769 - 0.774953 -1.88817
 1.07971  3.1227  1.43551  0.0216555  0.286909  0.902922
-1.41621  1.43551  0.690643  1.6702  1.87609  0.242296
-2.71769  0.0216555  1.6702  2.44206 -0.634897 -2.83436
-0.89372 -0.694587  0.713968 -0.450406  2.23828 -2.3204
-1.84011  0.733801 -2.02018 -2.64185 - 0.0940168  1.53355
 0.54638  1.359  1.24006  0.54343  1.01911  0.945984
-1.44939  0.827107  1.3198  0.679938  1.55354  0.347781
 1.54457 -0.837363 -1.31007 -1.56792 -0.271563 -1.07729
-0.215707  0.511466 -2.69192  0.26919  0.0336246 -0.319774
-4.31362  1.24963  1.25732  0.855534 -1.37662 -3.14565
 1.30665  2.03196 -2.52662  0.0660982 -0.633672  0.3982
 1.72119 -0.512198  1.22039  0.175069 -0.803392 -1.30454
 ⋮
-0.819445 -0.591849 -1.32706  2.69286  0.297423  0.300347
 2.79525  1.29027  3.35323 -2.14452 -0.942964 -2.7804
-0.760805  1.86249 -1.33621 -0.180748 - 2.03895 -0.160381
-0.685617 -0.902597 -1.35331 -3.1714 -0.675688  0.238703
-0.75208  0.0728725  2.87122 -0.714024  1.07543  0.0207712
 3.0112  2.11567  0.926898 -0.695955  0.666053 -1.65378
 0.495105  2.70711  3.27376  0.767181 -0.562891  2.13788
-1.10105  0.729133 -0.592985 -2.4141 - 0.519696 -0.634044
-2.23668 -2.9504  0.154699 -1.71485 -0.570891 -0.543797
-2.25884 -0.0762578 -0.4445 -0.556246 -1.38004  0.820376
-0.774953  0.286909  1.87609 -0.634897 -0.914585 -0.00694849
-1.88817  0.902922  0.242296 -2.83436 -0.00694849  1.08351

[70]: import Pkg
Pkg.add("BenchmarkTools")

Updating registry at '~/.julia/registries/General.toml'
Resolving package versions...
Installed BenchmarkTools v1.6.3
Updating '~/.julia/environments/v1.11/Project.toml'
[6e4b80f9] + BenchmarkTools v1.6.3
Updating '~/.julia/environments/v1.11/Manifest.toml'
[6e4b80f9] + BenchmarkTools v1.6.0 => v1.6.3
Precompiling project...
6482.3 ms ✓ BenchmarkTools
113992.3 ms ✓ MathOptInterface
9747.2 ms ✓ Optim v OptimMOIExt
29136.2 ms ✓ GLPK
73847.6 ms ✓ JuMP
5 dependencies successfully precompiled in 205 seconds. 646 already precompiled.

[71]: using BenchmarkTools

[72]: # Оценка эффективности выполнения операции по нахождению
# собственных значений симметризованной матрицы:
@btime eigvals(Asym);

407.268 ms (21 allocations: 7.99 MiB)

[73]: # Оценка эффективности выполнения операции по нахождению
# собственных значений зашумлённой матрицы:
@btime eigvals(Asym_noisy);

2.223 s (27 allocations: 7.93 MiB)

[74]: # Оценка эффективности выполнения операции по нахождению
# собственных значений зашумлённой матрицы,
# для которой явно указано, что она симметричная:
@btime eigvals(Asym_explicit);

410.174 ms (21 allocations: 7.99 MiB)

[75]: # Трёхдиагональная матрица 1000000 x 1000000:
n = 1000000;
A = SymTridiagonal(randn(n), randn(n-1))

[75]: 1000000x1000000 SymTridiagonal{Float64, Vector{Float64}}:
-1.28888 -1.2219  " " " " "
-1.2219 -1.55267 -0.708802 " " " "
" -0.708802  0.29243 " " " "
" " 0.652818 " " " "
" " " " " " "
" " " " " " "
" " " " " " "
" " " " " " "
" " " " " " "
" " " " " " "
```

Рис. 3.8: Оценка эффективности

Далее рассмотрим примеры работы с разреженными матрицами большой размерности (рис. 3.8). Использование типов `Tridiagonal` и `SymTridiagonal` для хранения трёхдиагональных матриц позволяет работать с потенциально очень большими трёхдиагональными матрицами (рис. 3.9):

```
[76]: # Оценка эффективности выполнения операции по нахождению
# собственных значений:
@time eigenvals(A)
636.392 ms (44 allocations: 183.11 MiB)
[76]: 6.237245657182745

[77]: B = Matrix(A)

OutOfMemoryError()
Stacktrace:
 [1] GenericMemory
   @ ./boot.jl:516 [inlined]
 [2] new_as_memoryref
   @ ./boot.jl:535 [inlined]
 [3] Array
   @ ./boot.jl:582 [inlined]
 [4] Matrix{Float64}(M::SymTridiagonal{Float64, Vector{Float64}})
   @ LinearAlgebra ~/julia/juliaup/julia-1.11.3+0.x64.linux.gnu/share/julia/stdlib/v1.11/LinearAlgebra/src/tridiag.jl:136
 [5] (Matrix)(M::SymTridiagonal{Float64, Vector{Float64}})
   @ LinearAlgebra ~/julia/juliaup/julia-1.11.3+0.x64.linux.gnu/share/julia/stdlib/v1.11/LinearAlgebra/src/tridiag.jl:147
 [6] top-level scope
   @ In[77]:1
```

Рис. 3.9: Оценка эффективности

При попытке задать подобную матрицу обычным способом и посчитать её собственные значения, вы скорее всего получите ошибку переполнения памяти.

3.6 Общая линейная алгебра

Обычный способ добавить поддержку числовой линейной алгебры - это обернуть подпрограммы BLAS и LAPACK. Собственно, для матриц с элементами `Float32`, `Float64`, `Complex{Float32}` или `Complex{Float64}` разработчики Julia использовали такое же решение. Однако Julia также поддерживает общую линейную алгебру, что позволяет, например, работать с матрицами и векторами рациональных чисел. Для задания рационального числа используется двойная косая черта: `1//2` В следующем примере показано, как можно решить систему линейных уравнений с рациональными элементами без преобразования в типы элементов с плавающей запятой (для избежания проблемы с переполнением используем `BigInt`) (рис. 3.10).

Общая линейная алгебра

```

[78]: 1//2
[78]: 1//2

[79]: # Матрица с рациональными элементами:
Arational = Matrix(Rational(BigInt))(rand(1:10, 3, 3))/10

[79]: 3x3 Matrix{Rational{BigInt}}:
 3//10  9//10  1//2
 2//5   3//10  1
 4//5   2//5   7//10

[80]: # Единичный вектор:
x = fill(1, 3)

[80]: 3-element Vector{Int64}:
 1
 1
 1

[81]: # Задаём вектор b:
b = Arational*x

[81]: 3-element Vector{Rational{BigInt}}:
 17//10
 17//10
 19//10

[82]: # Решая исходного уравнения получаем с помощью функции \
# убеждаемся, что x - единичный вектор:
Arational \ b

[82]: 3-element Vector{Rational{BigInt}}:
 1
 1
 1

[83]: # LU-разложение:
lu(Arational)

[83]: LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64}}
L factor:
3x3 Matrix{Rational{BigInt}}:
 1  0  0
 3//8 1  0
 1//2 2//15 1
U factor:
3x3 Matrix{Rational{BigInt}}:
 4//5  2//5   7//10
 0  3//4  19//80
 0  0  371//600

```

Рис. 3.10: Пример решения системы линейных уравнений с рациональными элементами

3.7 Задания для самостоятельного выполнения

3.7.1 Произведения векторов

1. Задайте вектор v . Умножьте вектор v скалярно сам на себя и сохраните результат в `dot_v` (рис. 3.11).
2. Умножьте v матрично на себя (внешнее произведение), присвоив результат переменной `outer_v` (рис. 3.11).

Задания для самостоятельного выполнения

```
[84]: # 4.4.1. Произведения векторов
#1
v = [1, 2, 3] # произвольный вектор-столбец
dot_v = dot(v, v) # скалярное произведение ( $v^T v$ )

[84]: 14

[85]: #2
outer_v = v * v^T #  $v^T$  — это транспонированный вектор-строка

[85]: 3x3 Matrix(Int64):
 1 2 3
 2 4 6
 3 6 9
```

Рис. 3.11: Пункт 1 и пункт 2

3.7.2 Системы линейных уравнений

1. Решить СЛАУ с двумя неизвестными (рис. 3.12):

```

# С двумя неизвестными
A1a = [1 1; 1 -1]
b1a = [2, 3]
println("A = $A1a, b = $b1a")
try
    x1a = A1a \ b1a
    println("Решение: x = ${x1a[1]}, y = ${x1a[2]}")
catch e
    println("Система не имеет единственного решения")
end

A = [1 1; 1 -1], b = [2, 3]
Решение: x = 2.5, y = -0.5

1: A1b = [1 1; 2 2]
b1b = [2, 4]
println("A = $A1b, b = $b1b")
println("Определитель A: ${det(A1b)}")
println("Ранг A: ${rank(A1b)}, ранг расширенной матрицы: ${rank([A1b b1b])}")
if rank(A1b) == rank([A1b b1b])
    println("Система совместна, имеет бесконечно много решений")
else
    println("Система несовместна")
end

A = [1 1; 2 2], b = [2, 4]
Определитель A: 0.0
Ранг A: 1, ранг расширенной матрицы: 1
Система совместна, имеет бесконечно много решений

1: A1b = [1 1; 2 2]
b1b = [2, 4]
println("A = $A1b, b = $b1b")
println("Определитель A: ${det(A1b)}")
println("Ранг A: ${rank(A1b)}, ранг расширенной матрицы: ${rank([A1b b1b])}")
if rank(A1b) == rank([A1b b1b])
    println("Система совместна, имеет бесконечно много решений")
else
    println("Система несовместна")
end

A = [1 1; 2 2], b = [2, 5]
Ранг A: 1, ранг расширенной матрицы: 2
Система несовместна - нет решений

1: A1d = [1 1; 2 2; 3 3]
b1d = [1, 2, 3]
println("A = $A1d, b = $b1d")
println("Ранг A: ${rank(A1d)}, ранг расширенной матрицы: ${rank([A1d b1d])}")
if rank(A1d) == rank([A1d b1d])
    println("Система совместна, имеет бесконечно много решений")
else
    println("Система несовместна")
end

A = [1 1; 2 2; 3 3], b = [1, 2, 3]
Ранг A: 1, ранг расширенной матрицы: 1
Система совместна, имеет бесконечно много решений

1: A1e = [1 1; 2 1; 1 -1]
b1e = [2, 1, 3]
println("A = $A1e, b = $b1e")
println("Ранг A: ${rank(A1e)}, ранг расширенной матрицы: ${rank([A1e b1e])}")
if rank(A1e) == rank([A1e b1e]) == 2
    println("Система совместна, имеет единственное решение")
    # Решаем методом наименьших квадратов для переопределенной системы
    x1e = A1e \ b1e
    println("Решение (метод наименьших квадратов)")
else
    println("Система не имеет точного решения")
end

A = [1 1; 2 1; 1 -1], b = [2, 1, 3]
Ранг A: 2, ранг расширенной матрицы: 3
Система не имеет точного решения

1: A1f = [1 1; 2 1; 3 2]
b1f = [2, 1, 3]
println("A = $A1f, b = $b1f")
println("Ранг A: ${rank(A1f)}, ранг расширенной матрицы: ${rank([A1f b1f])}")
if rank(A1f) == rank([A1f b1f]) == 2
    println("Система совместна, имеет единственное решение")
    x1f = A1f \ b1f
    println("Решение (метод наименьших квадратов): x = ${round(x1f[1], digits=3)}, y = ${round(x1f[2], digits=3)}")
else
    println("Система не имеет точного решения")
end

A = [1 1; 2 1; 3 2], b = [2, 1, 3]
Ранг A: 2, ранг расширенной матрицы: 2
Система совместна, имеет единственное решение
Решение (метод наименьших квадратов): x = -1.8, y = 3.8

```

Рис. 3.12: Решение СЛАУ с двумя неизвестными

2. Решить СЛАУ с тремя неизвестными (рис. 3.13).

```

1: # с тремя неизвестными

A2a = [1 1 1; 1 -1 -2]
b2a = [2, 3]
println("A = $A2a, b = $b2a")
println("Ранг A: $(rank(A2a))")
println("Система недоопределена, имеет бесконечно много решений")
println("Свободных переменных: $(3 - rank(A2a))")

A = [1 1 1; 1 -1 -2], b = [2, 3]
Ранг A: 2
Система недоопределена, имеет бесконечно много решений
Свободных переменных: 1

A2b = [1 1 1; 2 2 -3; 3 1 1]
b2b = [2, 4, 1]
println("A = $A2b, b = $b2b")
println("Определитель A: $(det(A2b))")
try
    x2b = A2b \ b2b
    println("Решение: x = $(round(x2b[1], digits=3)), y = $(round(x2b[2], digits=3)), z = $(round(x2b[3], digits=3))")
catch e
    println("Система не имеет единственного решения")
end

A = [1 1 1; 2 2 -3; 3 1 1], b = [2, 4, 1]
Определитель A: -18.0
Решение: x = -0.5, y = 2.5, z = 0.0

A2c = [1 1 1; 1 1 2; 2 2 3]
b2c = [1, 0, 1]
println("A = $A2c, b = $b2c")
println("Ранг A: $(rank(A2c)), ранг расширенной матрицы: $(rank([A2c b2c]))")
if rank(A2c) == rank([A2c b2c])
    if rank(A2c) == 3
        println("Система имеет единственное решение")
        x2c = A2c \ b2c
        println("Решение: x = $(x2c[1]), y = $(x2c[2]), z = $(x2c[3])")
    else
        println("Система совместна, имеет бесконечно много решений")
        println("Свободных переменных: $(3 - rank(A2c))")
    end
else
    println("Система несовместна")
end

A = [1 1 1; 1 1 2; 2 2 3], b = [1, 0, 1]
Ранг A: 2, ранг расширенной матрицы: 2
Система совместна, имеет бесконечно много решений
Свободных переменных: 1

A2d = [1 1 1; 1 1 2; 2 2 3]
b2d = [1, 0, 0]
println("A = $A2d, b = $b2d")
println("Ранг A: $(rank(A2d)), ранг расширенной матрицы: $(rank([A2d b2d]))")
if rank(A2d) == rank([A2d b2d])
    if rank(A2d) == 3
        println("Система имеет единственное решение")
        x2d = A2d \ b2d
        println("Решение: x = $(x2d[1]), y = $(x2d[2]), z = $(x2d[3])")
    else
        println("Система совместна, имеет бесконечно много решений")
    end
else
    println("Система несовместна - нет решений")
end

A = [1 1 1; 1 1 2; 2 2 3], b = [1, 0, 0]
Ранг A: 2, ранг расширенной матрицы: 3
Система несовместна - нет решений

```

Рис. 3.13: Решение СЛАУ с тремя неизвестными

3.7.3 Операции с матрицами

1. Приведите матрицы к диагональному виду (рис. 3.14).
2. Вычислите (рис. 3.14).

```

8): # 4.4.3. Операции с матрицами
# 1. Приведите приведенные ниже матрицы к диагональному виду
A1a = [1 -2; -2 1]
Diagonal(eigen(A1a).values)

8): 2x2 Diagonal(Float64, Vector{Float64}):
-1.0 0
0 3.0

3): A2a = [1 -2; -2 3]
Diagonal(eigen(A2a).values)

3): 2x2 Diagonal(Float64, Vector{Float64}):
-0.236068 0
0 4.236067

4): A3a = [1 -2 0; -2 1 2; 0 2 0]
Diagonal(eigen(A3a).values)

4): 3x3 Diagonal(Float64, Vector{Float64}):
-2.14134 0 0
0 0.515138 0
0 0 3.6262

9): # 2. Вычислите
A1a^10

9): 2x2 Matrix{Int64}:
29525 -29524
-29524 29525

2): B = [5 -2; -2 5]
sqrt(B)

2): 2x2 Matrix{Float64}:
2.1889 -0.45685
-0.45685 2.1889

0): A1a^(1/3)

0): 2x2 Symmetric{ComplexF64, Matrix{ComplexF64}}:
0.971125+0.433013im -0.471125+0.433013im
-0.471125+0.433013im 0.971125+0.433013im

1): K = [1 2; 2 3]
sqrt(K)

1): 2x2 Matrix{ComplexF64}:
0.568864+0.351578im 0.920442-0.217287im
0.920442-0.217287im 1.48931+0.134291im

7): 0 = [140 97 74 168 131;
97 106 89 131 36;
74 89 152 144 71;
168 131 144 54 142;
131 36 71 142 36]

7): 5x5 Matrix{Int64}:
140 97 74 168 131
97 106 89 131 36
74 89 152 144 71
168 131 144 54 142
131 36 71 142 36

```

Рис. 3.14: Приведение матриц к диагональному виду и вычисление

3. Найдите собственные значения матрицы A (рис. 3.14). Создайте диагональную матрицу из собственных значений матрицы A . Создайте нижнедиагональную матрицу из матрица A . Оцените эффективность выполняемых операций (рис. 3.15):

```

8]: @time G = eigen(Q)
0.000082 seconds (19 allocations: 3.047 KiB)
8]: Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
5-element Vector{Float64}:
-128.4932276488214
-55.887784553857845
42.75216727931888
87.16111477514491
542.4677381466137
vectors:
5x5 Matrix{Float64}:
-0.147575  0.647178  0.010882  0.548983 -0.507907
-0.256795 -0.173868  0.834628 -0.239864 -0.387253
-0.185537  0.239762 -0.422161 -0.731925 -0.440631
-0.819784 -0.247586 -0.8273194  0.8366447 -0.514526
-0.453885 -0.657619 -0.352577  0.322668 -0.364928

9]: @time J = Diagonal(G.values)
0.000027 seconds (1 allocation: 16 bytes)
9]: 5x5 Diagonal{Float64, Vector{Float64}}:
-128.493      "      "      "      "
"      -55.8878      "      "      "
"      "      42.7522      "      "
"      "      "      87.1611      "
"      "      "      "      542.468

10]: # нижнедиагональная матрица
@time T = LowerTriangular(Q)
0.000018 seconds (1 allocation: 16 bytes)
10]: 5x5 LowerTriangular{Int64, Matrix{Int64}}:
148      "      "      "      "
97 106      "      "      "
74 89 152      "      "
168 131 144 54      "
131 36 71 142 36

```

Рис. 3.15: Создание диагональной матрицы из собственных значений, ниже-диагональной матрицы, оценка эффективности

3.7.4 Линейные модели экономики

Линейная модель экономики может быть записана как СЛАУ: $x - Ax = y$, где элементы матрицы A и столбца y — неотрицательные числа. По своему смыслу в экономике элементы матрицы A и столбцов x , y не могут быть отрицательными числами (рис. 3.16)

```

151  sh  /1 142  sh
21]: # 4.4.4. Линейные модели экономики
# Функция для проверки продуктивности через критерий неотрицательности
function is_productive_inverse(A)
    n = size(A, 1)
    E = Matrix{Float64}(I, n, n)
    try
        inv_matrix = inv(E - A)
        return all(inv_matrix .>= 0)
    catch
        return false # Если матрица (E-A) вырождена
    end
end

21]: is_productive_inverse (generic function with 1 method)

22]: # Функция для проверки продуктивности через спектральный критерий
function is_productive_spectral(A)
    λ = eigen(A).values
    return all(abs.(λ) < 1)
end

22]: is_productive_spectral (generic function with 1 method)

29]: # Функция для проверки всех критериев
function check_productivity(A, name)
    println("Проверка матрицы $name:")
    println("A = ")
    println(A)

    inverse_productive = is_productive_inverse(A)
    spectral_productive = is_productive_spectral(A)

    println("Критерий (E-A)^-1 >= 0: $inverse_productive")
    println("Спектральный критерий |lambda| < 1: $spectral_productive")

    if inverse_productive == spectral_productive
        println("Оба критерия согласуются: матрица $(inverse_productive ? "продуктивна" : "не продуктивна")")
    else
        println("Критерии не согласуются!")
    end
    println()
end

29]: check_productivity (generic function with 1 method)

```

Рис. 3.16: Создание функций

1. Матрица A называется продуктивной, если решение x системы при любой неотрицательной правой части y имеет только неотрицательные элементы x_i . Используя это определение, проверьте, являются ли матрицы продуктивными (рис. 3.17).
2. Критерий продуктивности: матрица A является продуктивной тогда и только тогда, когда все элементы матрица: $(E - A)^{-1}$ являются неотрицательными числами. Используя этот критерий, проверьте, являются ли матрицы продуктивными (рис. 3.17).

```

0]: # 1
A1 = [1 2; 3 4]
A2 = 0.5 * [1 2; 3 4]
A3 = 0.1 * [1 2; 3 4]

check_productivity(A1, "A1 (исходная)")
check_productivity(A2, "A2 (×0.5)")
check_productivity(A3, "A3 (×0.1)")

Проверка матрицы A1 (исходная):
A =
[1 2; 3 4]
Критерий  $(E-A)^{-1} \geq 0$ : false
Спектральный критерий  $|\lambda| < 1$ : false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы A2 (×0.5):
A =
[0.5 1.0; 1.5 2.0]
Критерий  $(E-A)^{-1} \geq 0$ : false
Спектральный критерий  $|\lambda| < 1$ : false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы A3 (×0.1):
A =
[0.1 0.2; 0.30000000000000004 0.4]
Критерий  $(E-A)^{-1} \geq 0$ : true
Спектральный критерий  $|\lambda| < 1$ : true
Оба критерия согласуются: матрица продуктивна

1]: # 2
B1 = [1 2; 3 1]
B2 = 0.5 * [1 2; 3 1]
B3 = 0.1 * [1 2; 3 1]

check_productivity(B1, "B1 (исходная)")
check_productivity(B2, "B2 (×0.5)")
check_productivity(B3, "B3 (×0.1)")

Проверка матрицы B1 (исходная):
A =
[1 2; 3 1]
Критерий  $(E-A)^{-1} \geq 0$ : false
Спектральный критерий  $|\lambda| < 1$ : false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы B2 (×0.5):
A =
[0.5 1.0; 1.5 0.5]
Критерий  $(E-A)^{-1} \geq 0$ : false
Спектральный критерий  $|\lambda| < 1$ : false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы B3 (×0.1):
A =
[0.1 0.2; 0.30000000000000004 0.1]
Критерий  $(E-A)^{-1} \geq 0$ : true
Спектральный критерий  $|\lambda| < 1$ : true
Оба критерия согласуются: матрица продуктивна

2]: # Те же матрицы B1, B2, B3 плюс новая

```

Рис. 3.17: Проверка с помощью определения и критерия на продуктивность

3. Спектральный критерий продуктивности: матрица A является продуктивной тогда и только тогда, когда все её собственные значения по модулю меньше 1. Используя этот критерий, проверьте, являются ли матрицы продуктивными (рис. 3.18).

```

# Те же матрицы B1, B2, B3 плюс новая
C1 = [1 2; 3 1]
C2 = 0.5 * [1 2; 3 1]
C3 = 0.1 * [1 2; 3 1]
C4 = [0.1 0.2 0.3; 0 0.1 0.2; 0 0.1 0.3]

check_productivity(C1, "C1 (исходная)")
check_productivity(C2, "C2 (×0.5)")
check_productivity(C3, "C3 (×0.1)")
check_productivity(C4, "C4 (3×3 матрица)")

Проверка матрицы C1 (исходная):
A =
[1 2; 3 1]
Критерий (E-A)^-1 >= 0: false
Спектральный критерий |lambda| < 1: false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы C2 (×0.5):
A =
[0.5 1.0; 1.5 0.5]
Критерий (E-A)^-1 >= 0: false
Спектральный критерий |lambda| < 1: false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы C3 (×0.1):
A =
[0.1 0.2; 0.30000000000000004 0.1]
Критерий (E-A)^-1 >= 0: true
Спектральный критерий |lambda| < 1: true
Оба критерия согласуются: матрица продуктивна

Проверка матрицы C4 (3×3 матрица):
A =
[0.1 0.2 0.3; 0.0 0.1 0.2; 0.0 0.1 0.3]
Критерий (E-A)^-1 >= 0: true
Спектральный критерий |lambda| < 1: true
Оба критерия согласуются: матрица продуктивна

```

Рис. 3.18: Проверка с помощью спектрального критерия

4 Выводы

В результате выполнения данной лабораторной работы я изучила возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.